

CS 571: Programming Languages

Assignment 3 (Coding). Due: March 13, 11:59PM

This assignment is adapted from Kenneth C. Loudon, Programming Languages Principles and Practice 2nd Edition Copyright (C) Brooks-Cole/ITP, 2003.

Project Description: Recursive-descent is a parsing technique that turns the nonterminals of production rules into groups of mutually recursive procedures whose actions are based on the right-hand sides of the rules. The right-hand sides are interpreted in the procedures as follows:

- Terminals are matched directly with input tokens as constructed by a scanner.
- Nonterminals are interpreted as calls to the procedures corresponding to the nonterminals.

Recursive-descent parsers are easy to implement manually because the parsing logic naturally mirrors the structure of the grammar. However, one major limitation is that they cannot handle grammars with left recursion, because such rules lead to infinite loops. Consider the following grammar, written in **Backus-Naur form (BNF)**, for simple non-negative integer arithmetic expressions used by a calculator:

```
expr → expr + term | term
term → term * factor | factor
factor → (expr) | number
number → number digit | digit
digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

The rule for `expr` is left-recursive, making the `+` operator left-associative. This rule generates an expression as a sequence of terms separated by `+` signs:

```
expr ⇒ expr + term
      ⇒ expr + term + term
      ⇒ expr + term + term + term
      ...
      ⇒ term + ... + term
```

This situation occurs very frequently, so a special notation for such grammar rules is adopted that expresses more clearly the repetitive nature of their structures:

$$\text{expr} \rightarrow \text{term} \{ + \text{term} \}$$

In this notation, the curly brackets $\{\square\}$ stand for zero or more repetitions of $\square$. Thus, the curly brackets have become new meta-symbols, and this notation is called **extended Backus-Naur form (EBNF)**. With this notation, we can rewrite the grammar above as:

```
expr → term { + term }
term → factor { * factor }
factor → (expr) | number
number → digit { digit }
digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

This form of the grammar rule supports recursive-descent parsing. For example, the nonterminal `expr` corresponds to the follow code:

```
1 int expr() {
2     int result = term();
3     while (token == '+') {
4         match('+');
5         result += term();
6     }
7     return result;
8 }
```

Thus, the curly brackets in EBNF represent **left recursion removal** by the use of a loop. The sample code [arith.c](#) provides a complete implementation of a calculator that can evaluate arithmetic expressions as defined by the grammar above.

Your Task: For this assignment, your goal is to extend the calculator by adding support for a new operator $\wedge$, which represents exponentiation (power). For example, the expression 2^3 should evaluate to $2^3 = 8$, where first integer is the **base**, and the second integer is the **exponent**. To accomplish this, you need to complete the following two tasks:

Task 1. (Not Graded) Update the grammar to include a new terminal $\wedge$ for the power operator with the following rules:

- **Right-associativity:** 2^3^2 should be evaluated as $2^{(3^2)}$ which equals $2^9 = 512$.
- **Precedence:** $\wedge$ has higher precedence than $*$ and $+$, but lower precedence than parentheses $()$. For example, $2*2^3$ should be evaluated as $2*(2^3)$ and result in 16.

Task 2. Modify the provided code to implement a parser that supports the updated grammar with the new $\wedge$ operator. Small adjustments to the existing code should be sufficient.

Testing and Auto Grading: We will use an automated script to grade your assignment. You can test your code locally by running: [make grade](#). Note that the autograder includes **additional test cases** beyond what you see in the script provided.

Submission: Compress your code into a file named [hw3.zip](#) containing a single source file [arith.c](#) with **your modifications**, then upload the ZIP file to Brightspace. You can use the following command to prepare your submission: [make submit](#).